

MATLAB

2. Vektory a matice



Stredná priemyselná škola elektrotechnická, Karola Adlera 5

Čo nás dnes čaká

1. Vektory a matice — čo to je a aké majú rozmery
2. Vytváranie vektorov a matíc
3. Rozmery poľa: `size`, `length`, `numel`
4. Indexovanie, zmena a mazanie prvkov
5. Skalárne a prvkové operácie — a prečo tam patrí bodka
6. Maticové násobenie
7. Logické vektory a logické indexovanie
8. Riešenie sústav lineárnych rovníc

Vektory a matice — teória

- MATLAB = **MA**TriX **LAB**oratory — všetko je postavené na poliach
- **Matica** je tabuľka hodnôt s rozmermi $r \times c$ (r riadkov, c stĺpcov)
- **Riadkový vektor** má rozmer $1 \times n$, **stĺpcový vektor** $n \times 1$
- **Skalár** (jedno číslo) je matica 1×1 — vektory aj skaláre sú teda len špeciálne prípady matice
- Jednotlivé hodnoty voláme **prvky** (elements)
- Všetky prvky poľa musia byť **rovnakého typu**

Riadkové vektory — teória

- Hodnoty zapíšeme do **hranatých zátvoriek**, oddelené medzerou alebo čiarkou
- Ak sú hodnoty pravidelne rozložené, použijeme **operátor dvojbodky**:
`zaciatok:koniec`
- S krokom: `zaciatok:krok:koniec` — krok môže byť aj **záporný** (klesajúca postupnosť)
- Pri dvojbodke **netreba** hranaté zátvorky
- `linspace(x,y,n)` vytvorí n rovnomerne rozložených hodnôt od x po y (vrátane oboch)
- `logspace(x,y,n)` to isté, ale logaritmicky — od 10^x po 10^y

Riadkové vektory — príklad

```
v = [1 2 3 4]
vec = 2:6
nv = 1:2:9
zost = 9:-2:1
ls = linspace(3,15,5)
```

```
v =
     1     2     3     4

vec =
     2     3     4     5     6

nv =
     1     3     5     7     9

zost =
     9     7     5     3     1

ls =
     3     6     9    12    15
```

Pozor: `1:2:6` dá 1 3 5 — krok by presiahol koniec, tak sa postupnosť zastaví.

Stĺpcové vektory a transpozícia

```
c = [1; 2; 3; 4]
r = 1:3;
ct = r'
```

Prvky oddelené **bodkočiarkou** tvoria stĺpcový vektor.

Apostrof **'** je operátor **transpozície** — vymení riadky za stĺpce.

```
c =
     1
     2
     3
     4

ct =
     1
     2
     3
```

Matice — teória

- Riadky matice oddelujeme **bodkočiarkou**: `[1 2 3; 4 5 6]`
- Každý riadok musí mať rovnaký počet prvkov — inak MATLAB ohlásí chybu
- Hotové matice vieme vytvoriť funkciami:
 - `zeros(m,n)` — samé nuly
 - `ones(m,n)` — samé jednotky
 - `eye(n)` — jednotková matica (jednotky na diagonále)
 - `rand(m,n)`, `randi(max,m,n)` — náhodné čísla
- Ak zadáme len jedno číslo, dostaneme štvorcovú maticu: `ones(2)` je 2×2

Matice – příklad

```
mat = [1 2 3; 4 5 6]  
Z = zeros(2,3)  
O = ones(2)  
I3 = eye(3)
```

```
mat =  
    1    2    3  
    4    5    6  
  
Z =  
    0    0    0  
    0    0    0  
  
O =  
    1    1  
    1    1  
  
I3 =  
    1    0    0  
    0    1    0  
    0    0    1
```

Rozmery poľa – teória

- `size(A)` vráti **počet riadkov a stĺpcov** (v tomto poradí)
- Obe hodnoty vieme zachytiť naraz: `[r, c] = size(A)` — funkcia vracia **viac výstupov**
- `length(A)` vráti **najväčší** z rozmerov (pri vektore jeho dĺžku)
- `numel(A)` vráti **celkový počet prvkov** ($r \cdot c$)
- Dobrý zvyk: nikdy nepredpokladajme rozmery natvrdo — radšej ich zistíme funkciou

Rozmery poľa – príklad

```
mat = [1 2 3; 4 5 6];  
size(mat)  
[rr, cc] = size(mat)  
length(mat)  
numel(mat)
```

```
ans =  
     2     3  
  
rr =  
     2  
  
cc =  
     3  
  
ans =  
     3  
  
ans =  
     6
```

Indexovanie – teória

- Poradové číslo prvku sa volá **index**; v MATLABe sa začína **od 1**
- K prvku pristupujeme cez **okružle zátvorky**: $v(3)$, pri matici $M(\text{riadok}, \text{stlpec})$
- Dvojbodkou vyberieme rozsah: $v(2:4)$; samotná $:$ znamená „všetky“
- $M(2,:)$ je celý druhý riadok, $M(:,1)$ je celý prvý stĺpec
- Indexom môže byť aj **vektor indexov**: $v([1\ 10\ 5])$
- **end** označuje posledný index v danom rozmere

Indexovanie – príklad

```
newvec = [1 3 5 7 9 3 ...  
          6 9 12 15];  
newvec(5)  
newvec(4:6)  
newvec([1 10 5])  
mat(2,:)   
mat(:,1)
```

```
ans =  
     9  
  
ans =  
     7     9     3  
  
ans =  
     1    15     9  
  
ans =  
     4     5     6  
  
ans =  
     1  
     4
```

Zmena a mazanie prvkov — teória

- Prvok zmeníme priradením na jeho index: $b(2) = 11$
- Priradením za koniec sa vektor **predĺži**: z troch prvkov spravíme štyri
- Ak vznikne medzera, MATLAB ju vyplní **nulami**
- Prvok (alebo celý riadok/stĺpec) **zmažeme** priradením prázdnej matice $[]$
- $d(2) = []$ zmaže druhý prvok, $M(2,:) = []$ celý druhý riadok
- Predlžovanie polí v cykle je **pomalé** — lepšie je pole vopred pripraviť napr. cez `zeros`

Zmena a mazanie prvkov — príklad

```
b = [7 9 3];  
b(2) = 11  
rv = [3 55 11];  
rv(4) = 2  
d = [10 20 30 40 50];  
d(2) = []
```

```
b =  
    7    11     3  
  
rv =  
    3    55    11     2  
  
d =  
   10    30    40    50
```

Rovnako $M(2,:) = []$ odstráni z matice celý druhý riadok.

Zmena tvaru a spájanie — teória

- `reshape(A,m,n)` zmení rozmery poľa — prvky berie **po stĺpcoch**
- Počet prvkov musí sedieť: z 2×6 vieme spraviť 3×4 , ale nie 3×3
- `flipplr` prevráti pole zľava doprava, `flipud` zhora nadol, `rot90` otočí o 90°
- **Spájanie (concatenation):** `[A B]` vedľa seba, `[A; B]` pod seba
- Pri spájaní musia sedieť rozmery — vedľa seba rovnaký počet riadkov, pod seba rovnaký počet stĺpcov

Zmena tvaru a spájanie — príklad

```
reshape(1:6,2,3)
```

```
A1 = [1 2; 3 4];
```

```
B1 = [5 6; 7 8];
```

```
[A1 B1]
```

```
[A1; B1]
```

```
ans =
```

```
1    3    5
```

```
2    4    6
```

```
ans =
```

```
1    2    5    6
```

```
3    4    7    8
```

```
ans =
```

```
1    2
```

```
3    4
```

```
5    6
```

```
7    8
```

Všimnite si: `reshape` napĺňa novú maticu **po stĺpcoch**, nie po riadkoch.

Skalárne a prvkové operácie – teória

- **Skalárna operácia:** pole $\times$ číslo — vykoná sa na **každom prvku** ($v*3$, $v/2$)
- **Prvkové operácie** pracujú prvok po prvku, takže obe polia musia mať **rovnaké rozmery**
- Sčítanie a odčítanie sú prvkové automaticky: $v1 + v2$
- **Ale pri násobení, delení a mocnení treba BODKU:** $.*$, $./$, $.^$
- Bez bodky by MATLAB rozumel **maticovú** operáciu — to je niečo úplne iné

Skalárne a prvkové operácie – príklad

```
w = [3 7 2 1];  
w * 3  
w .^ 2  
v1 = 2:5;  
v2 = [33 11 5 1];  
v1 .* v2  
w ^ 2
```

```
ans =  
     9     21     6     3
```

```
ans =  
     9     49     4     1
```

```
ans =  
    66    33    20     5
```

```
Error using ^  
Incorrect dimensions for  
raising a matrix to a power.  
... use POWER (.^) for  
elementwise power.
```

$w \wedge 2$ by znamenalo $w \cdot w$ ako matice — to pre riadkový vektor nedáva zmysel, preto chyba.

Maticové násobenie – teória

- Maticové násobenie **nie je** násobenie prvok po prvku – má vlastné pravidlo
- Aby sme mohli násobiť $A \cdot B$, musia sa rovnať **vnútorné rozmery**:

$$(A)_{m \times n} \cdot (B)_{n \times p} = (C)_{m \times p}$$

- Výsledok má rozmery **vonkajšie**: toľko riadkov ako A a toľko stĺpcov ako B
- Prvok c_{ij} je súčet súčinov i -teho riadku A a j -teho stĺpca B
- Preto **nie je** komutatívne: $A \cdot B \neq B \cdot A$ (a často ani nie je definované)

Maticové násobenie — príklad

```
A = [3 8 0; 1 2 5];  
B = [1 2 3 1;  
     4 5 1 2;  
     0 2 3 0];  
C = A*B
```

A je 2×3 , B je 3×4 — vnútorné rozmery
(3) sedia, výsledok je 2×4 .

Napr. $C(1,1) = 3 \cdot 1 + 8 \cdot 4 + 0 \cdot 0 = 35$.

```
C =  
35    46    17    19  
9     22    20     5
```

Logické vektory – teória

- Porovnanie celého poľa dá **logické pole** rovnakých rozmerov: `vec > 5`
- Toto pole vieme použiť ako index – hovoríme tomu **logické indexovanie**
- `vec(vec > 5)` vráti len tie prvky, kde je podmienka splnená
- `sum(...)` na logickom poli spočíta, **koľko** prvkov podmienku spĺňa
- `find(...)` vráti **indexy** prvkov, ktoré podmienku spĺňajú
- `any` = spĺňa aspoň jeden, `all` = spĺňajú všetky

Logické vektory – príklad

```
vecl = [5 9 3 4 6 11];  
isg = vecl > 5  
sum(isg)  
vecl(isg)  
find(vecl > 5)
```

```
isg =  
1x6 logical array  
0 1 0 0 1 1  
  
ans =  
3  
  
ans =  
9 6 11  
  
ans =  
2 5 6
```

`vecl(isg)` vráti hodnoty, `find` vráti indexy – to je ten rozdiel.

Sústavy lineárnych rovníc – teória

- Sústavu rovníc zapíšeme maticovo ako $A \cdot x = b$
- A je matica koeficientov, b stĺpcový vektor pravých strán, x hľadané riešenie
- V MATLABe ju vyriešime **ľavým delením**: $x = A \setminus b$
- Matematicky by sme písali $x = A^{-1}b$, teda $x = \text{inv}(A) * b$ – funguje tiež
- **Odporúča sa však $A \setminus b$** – je rýchlejšie a presnejšie než počítat inverznú maticu

Sústavy lineárnych rovníc – príklad

$$2x_1 + 5x_2 + 3x_3 = 1 \quad 4x_1 + 6x_2 + 2x_3 = 5 \quad x_1 - 5x_2 + 3x_3 = 3$$

```
A = [2 5 3; 4 6 2; 1 -5 3];  
b = [1; 5; 3];  
x = A \ b
```

```
x =  
    2.0278  
   -0.4028  
   -0.3472
```

Riešenie je $x_1 = 2,03$, $x_2 = -0,40$, $x_3 = -0,35$. Koeficienty čítame po riadkoch presne tak, ako sú v sústave.

Časté chyby — na čo si dať pozor

- Matica, ktorej riadky nemajú **rovnaký počet prvkov** — skončí chybou
- Zámena `*` a `.*` — maticové vs. prvkové násobenie sú úplne odlišné operácie
- Zabudnutá **bodka** pri `.*`, `./`, `.^`
- Indexovanie poľom čísel 0/1 typu `double` — na indexovanie musí byť typ `logical`
- Použitie `&&` alebo `||` na poliach — pre polia patria `&` a `|`
- Predpokladanie rozmerov natvrdo namiesto `size` / `numel`

Zhrnutie

- **Dnes:** vektory a matice, ich rozmery, indexovanie od 1, zmena a mazanie prvkov, prvkové vs. maticové operácie, logické indexovanie a sústavy rovníc
- **Zapamätajte si:** bodka pri `.*`, `./`, `.^` robí operáciu **prvok po prvku**; bez bodky ide o **maticovú** operáciu

Ďakujem za pozornosť

Otázky a diskusia
